Universidad Da Vinci de Guatemala

Facultad de ingeniería en sistemas

Curso: Fundamentos de Programación

Catedraditco : Brandon Chitay

Roberto Estuardo Chavarria Zelada

Carnet: 202504060

Fecha: 22 de febrero de 2026

INTRODUCCIÓN

En el presente trabajo se realiza un análisis empírico del tiempo de ejecución de distintos algoritmos implementados en Java, con el objetivo de comparar su comportamiento práctico con su complejidad teórica.

Se midieron los tiempos promedio en nanosegundos utilizando la función System.nanoTime(), ejecutando múltiples repeticiones para obtener resultados más estables. Los datos obtenidos fueron exportados a archivos CSV y posteriormente graficados en Excel para visualizar el crecimiento del tiempo de ejecución en función del tamaño de entrada (n).

Los algoritmos analizados fueron:

- **Cálculo de factorial (iterativo y recursivo)**
- **Fibonacci (iterativo y recursivo)**
- **Búsqueda lineal (iterativa y recursiva)**
- **Bubble Sort (iterativo y recursivo)**

Este análisis permite observar cómo el crecimiento del tiempo de ejecución confirma la teoría de complejidad computacional estudiada en clase.

Este análisis permite observar cómo el crecimiento del tiempo de ejecución confirma la teoría de complejidad computacional estudiada en clase.

1. **ANÁLISIS DEL ALGORITMO FACTORIAL**

Descripción

El algoritmo factorial calcula el producto de todos los números enteros positivos hasta n. Se implemento tanto de forma iterativa como recursiva

Complejidad Teórica

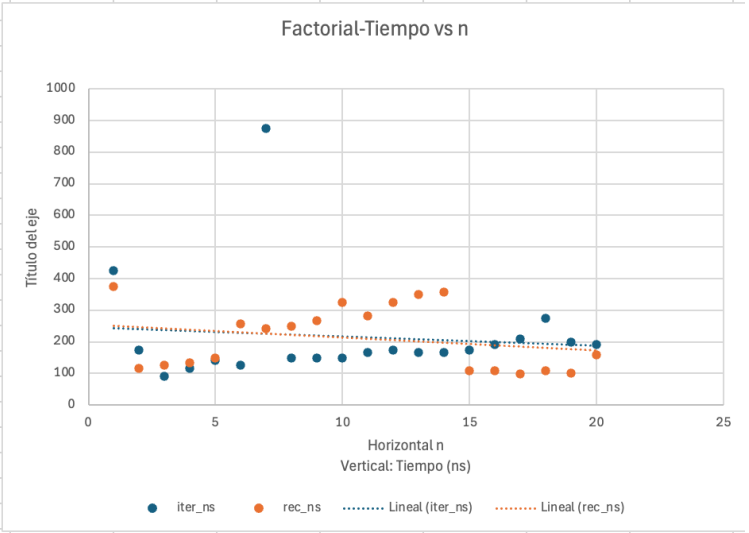
Iterativo: O (n)

Recursivo: O(n)

Ambas versiones realizan exactamente n operaciones, por lo que su crecimiento es lineal.

Análisis de la grafica

n	iter_ns	rec_ns
1	425	375
2	175	117
3	91	125
4	116	133
5	141	149
6	125	258
7	875	241
8	150	250
9	149	266
10	150	325
11	166	283
12	175	325
13	166	349
14	166	358
15	175	108
16	191	108
17	208	99
18	275	108
19	200	100
20	191	158



Se observa un crecimiento lineal en ambas implementaciones. La versión recursiva presenta una ligera variación debido al costo adicional de las llamadas en pila de ejecución,

En conclusión, el comportamiento experimental confirma que el algoritmo factorial posee complejidad lineal, ya que el tiempo de ejecución aumenta proporcionalmente al valor de n.

2. **Análisis del algoritmo Fibonacci**

Descripción

El algoritmo Fibonacci calcula el termino n de la secuencia de Fibonacci. Se implemento tanto de forma iterativa como recursiva.

Complejidad Teórica

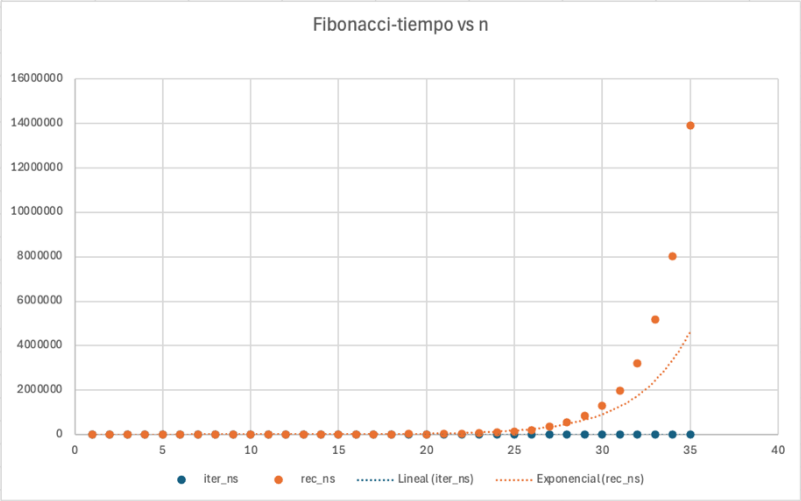
Iterativo: O(n)

Recursivo: O (2”)

La versión recursiva realiza múltiples llamadas repetidas, lo que provoca crecimiento exponencial.

Análisis de la Grafica

n	iter_ns	rec_ns
1	266	266
2	141	116
3	108	175
4	91	266
5	108	375
6	108	625
7	125	1433
8	133	1591
9	200	2350
10	141	3708
11	141	541
12	183	841
13	191	1199
14	183	1950
15	191	3200
16	200	5000
17	191	8341
18	291	13241
19	200	16625
20	233	12908
21	224	19916
22	225	32208
23	241	52566
24	224	84133
25	1250	136808
26	174	210191
27	91	341741
28	91	535633
29	91	841333
30	100	1294175
31	91	1982908
32	66	3218041
33	141	5168491
34	116	8021800
35	149	13907608



En la gráfica se observa que la versión recursiva crece de manera exponencial, mientras que la versión iterativa mantiene un crecimiento lineal.

En conclusión, los resultados experimentales confirman que la implementación recursiva de Fibonacci es altamente ineficiente debido a su crecimiento exponencial.

3. **Análisis de búsqueda lineal**

Descripción

La Búsqueda lineal recorre un arreglo hasta encontrar el elemento deseado o llegar al final,

Complejidad Teórica

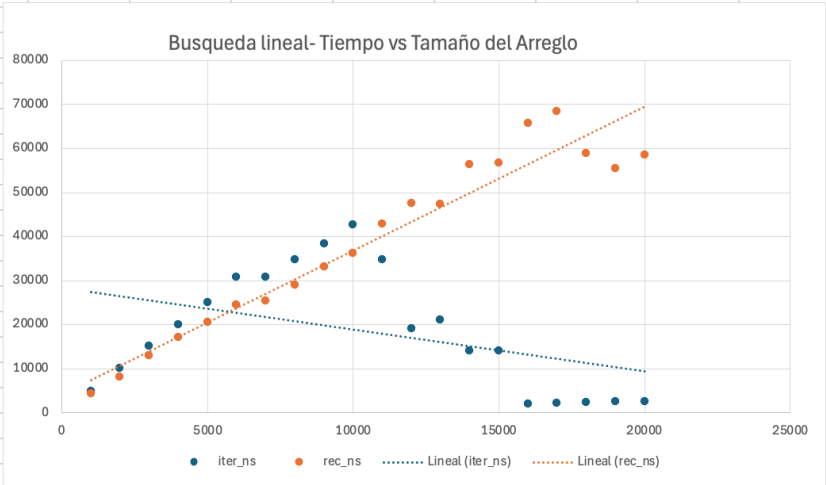
Iterativa: O(n)

Recursiva O(n)

En el peor caso, el algoritmo revisa todos los elementos del arreglo.

Análisis de la grafica

n	iter_ns	rec_ns
1000	5016	4425
2000	10241	8125
3000	15125	13016
4000	20100	17166
5000	25075	20516
6000	30916	24591
7000	30791	25450
8000	34733	29025
9000	38475	33158
10000	42641	36200
11000	34741	42958
12000	19241	47608
13000	21183	47474
14000	14183	56449
15000	14199	56741
16000	2125	65708
17000	2258	68533
18000	2391	58916
19000	2516	55508
20000	2641	58550



La grafica muestra el crecimiento lineal conformo aumenta el tamaño del arreglo, la búsqueda lineal confirma su complejidad O(n), ya que el tiempo aumenta proporcionalmente al tamaño del arreglo

4. **Análisis de bubble sort**

Descripcion

Bubble Sort ordena un arreglo comparado elementos adyacentes repetidamente.

Complejidad Teorica

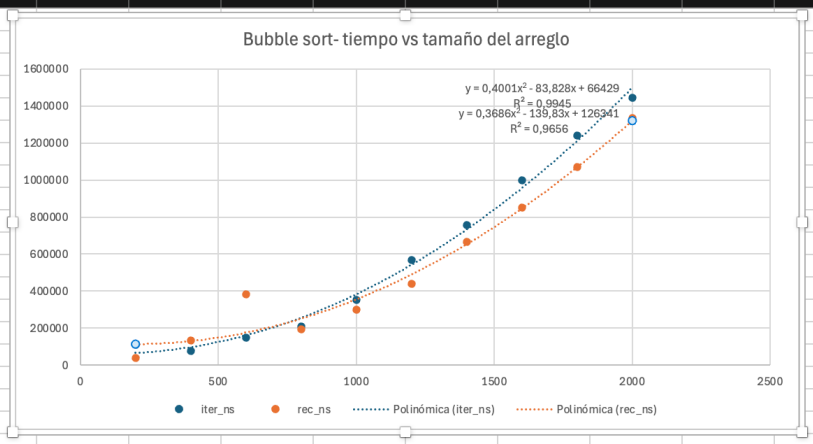
Iterativo: $O(n^2)$

Recursivo: O(n²)

Se realiza múltiples comparaciones anidadas, lo que genera crecimiento cuadráticos

Analisis de la Grafica

n	iter_ns	rec_ns
200	108975	38083
400	75116	132158
600	149408	380358
800	209858	193391
1000	351974	298475
1200	568341	439449
1400	756050	664258
1600	999366	849216
1800	1240275	1071849
2000	1444741	1335083



En conclusión, el comportamiento experimental confirma que Bumbble Sort posee complejidad O(n²), siendo considerablemente más lento conforme aumenta el tamaño del arreglo.

Conclusión General

El análisis empírico realizado demuestra que los tiempos de ejecución obtenidos experimentalmente coinciden con la complejidad teórica de cada algoritmo.

Se comprobó que:

- **Los algoritmos lineales presentan crecimiento proporcional.**
- **Los algoritmos cuadráticos muestran incrementos acelerados.**
- **Los algoritmos exponenciales crecen de manera extremadamente rápida.**
- **La recursividad puede implicar un costo adicional significativo dependiendo del algoritmo.**

Este estudio permitió comprender de manera práctica la importancia de la eficiencia algorítmica y cómo la elección del algoritmo impacta directamente en el rendimiento del sistema.